

Billing Software

Documentation

Version 0.1.0

7 November 2026

https://github.com/Akshat-Shah04/Billing-Software

Contents

Overview

- Description
- Features (v0.1)
- Tech Stack
- Project structure
- Documentation
- Installing on a new computer (Windows)
- Getting started (for development)
- Key design rules
- Look and feel
- Client folders
- Known limitations (ideas for v0.2)

User guide

- 1. Logging in
- 2. Clients
- 3. Services
- 4. Creating a bill
- 5. Finding, viewing, editing and printing bills
- 6. Staff accounts (admin)
- 7. Organisation settings (admin)
- 8. Client folders on the PC
- 9. Backups and restoring
- 10. Light and dark theme
- 11. Troubleshooting

API reference

- Conventions
- Health
- Authentication
- Users (admin)
- Clients
- Services
- Bills
- Settings
- Client folders (admin)
- Example

Architecture and development

- Overview
- Repository layout

Data model
Database adapter
Client folders
Backups
Security notes
Configuration
Development
Installer and releases
Known limitations

Decision records

1. Bill search
2. Bills can be edited in v0.1
3. GST is built, but off by default
4. Admin creates staff accounts
5. Money is stored as integer paise
6. Soft delete for clients, services and staff
7. Windows installer: one office PC as the server, embedded database
9. A readable folder per client, kept on the PC
10. Financial years and service descriptions on bills
11. Logo and payment details on every bill
12. Themes and design principles
8. Plain SQL and a small database adapter

Billing Software

Developer: Akshat Shah

Status: v0.1 feature-complete (backend, frontend and automated tests)

Description

A billing software for an accountancy firm to manage clients, services and bills.

Features (v0.1)

- Login and logout (no public sign-up: an admin creates the staff accounts)
- Staff management (admin): create accounts, reset passwords, deactivate / reactivate
- Add client, update client details (including **Referred by**), delete client (soft delete, with restore)
- A **folder per client on the PC**, named after the client, holding their details and a printable copy of every bill (updated automatically; see "Client folders" below)
- Add service (with a description), update service, deactivate service
- Create bill with several line items and a live total preview. Each line shows the service's description as entered in the service, copied onto the bill
- Update bill
- Search bills by client name or bill number, newest first, paginated, with a **financial year dropdown** (Indian FY, April to March, e.g. 2026-27)
- Print-friendly bill view (prints without the browser's date, title and address header)
- Organisation settings (admin): firm details, **logo upload**, **payment details** (bank account, IFSC, UPI ID and instructions, so clients know where to pay), and a GST on/off toggle with a configurable rate. The logo and payment details print on every bill and in the client-folder copies

Tech Stack

Area	Technology
Language	TypeScript
Frontend	React (with Vite), React Router
Backend	Node.js + Express 5, Zod validation
Database	PostgreSQL (plain SQL migrations, `pg` driver)
Auth	JWT + bcrypt, roles: admin / staff
Tests	Vitest + Supertest
Version control	Git + GitHub
Project management	GitHub Projects

Project structure

```
billing-software/
├── server/              Express API
│   ├── migrations/      SQL files, applied in order at startup
│   ├── src/routes/      auth, users, clients, services, settings, bills
│   ├── src/lib/         money maths, paging, error handling
│   ├── src/scripts/     seed.ts (creates the first admin), restore.ts (restores a backup)
│   └── test/            automated tests
├── client/              React app
│   └── src/pages/       one file per screen
├── installer/           Windows installer (Inno Setup script, build script, wizard text)
├── .github/workflows/   CI tests and the release build
└── docs/decisions.md    why things are the way they are
```

Documentation

Document	For
User guide	Staff and administrators: every screen, step by step, backups and troubleshooting
API reference	Every endpoint, field limit, status code and the GST/financial-year rules
Architecture and development	How it is built, data model, configuration, tests, installer and releases
Decision records	Why each important choice was made

PDF copies of all of these are in `docs/pdf`: one PDF per document, plus `Billing-Software-Documentation.pdf`, which has everything in one file with a cover page and clickable contents. They are built from the Markdown files, so after editing the docs regenerate them (needs Microsoft Edge or Google Chrome installed):

```
cd docs/pdf-build
npm install
npm run build
```

Installing on a new computer (Windows)

For an office, use the installer. It needs nothing installed beforehand.

1. Open the project's **Releases** page on GitHub and download `BillingSoftware-Setup-<version>.exe` (the `.sha256` file next to it lets you verify the download).
2. Run it. Windows may say "unknown publisher" because the installer is not code-signed: choose **More info > Run anyway**.
3. The wizard explains what it will do, then asks for a network port (default 4000) and the first administrator's name, username and password. It shows a full summary before it installs anything.
4. When it finishes, the software is already running as a Windows service. Open `http://localhost:4000` on that PC, and `http://<that-pc-name>:4000` from the other office PCs.

What the installer sets up: the program in `C:\Program Files\Billing Software`, a Windows service that starts with Windows, a firewall rule for private/domain networks only, and a data folder `C:\ProgramData\BillingSoftware` holding the database, daily backups (last 14 kept) and logs. Running the

installer again upgrades the program and keeps all data. Restoring a backup: `"C:\Program Files\Billing Software\tools\restore.cmd" "<backup file>"` from an administrator prompt. The wizard's own pages (`before`, `after`) cover this in more detail.

Publishing a new installer (for the developer)

```
git tag v0.1.0
git push origin v0.1.0
```

The workflow in `.github/workflows/release.yml` runs the tests, builds the app, bundles Node.js and the service wrapper, compiles the installer with Inno Setup and attaches it to a GitHub Release. To build locally, run `installer\build.ps1`, then compile `installer\billing.iss` with Inno Setup 6.3+.

Getting started (for development)

You need Node.js 22 or newer. For a real database you also need PostgreSQL.

1. Backend

```
cd server
npm install
cp .env.example .env
```

Open `server/.env` and set:

- `DATABASE_URL`. Use your PostgreSQL connection string, for example `postgres://postgres:YOURPASSWORD@localhost:5432/billing` (create an empty `billing` database first). The default, `pglite:./.data`, is an embedded Postgres that needs no setup, which is handy for a quick try.
- `JWT_SECRET`. Any long random string.
- `ADMIN_USERNAME` / `ADMIN_PASSWORD`. The first admin account. Change the password from the example.

Then create the tables and the first admin, and start the API:

```
npm run seed
npm run dev
```

The API runs on http://localhost:4000.

2. Frontend

In a second terminal:

```
cd client
npm install
npm run dev
```

Open http://localhost:5173 and log in with the admin you seeded.

3. Tests

```
cd server
npm test
```

The tests use an in-memory Postgres (PGlite), so they need no database setup.

Running as one server

`npm run build` in `client/` creates `client/dist`. When that folder exists, the Express server serves it too, so the whole app runs from http://localhost:4000.

Key design rules

- **Money is whole paise** (integer), never floating point. The screens convert to rupees.
- **GST is a snapshot.** A bill stores the rate and amount it was created with. Changing the setting later never changes old bills, and editing a bill recalculates with that bill's own rate.
- **Soft delete** for clients and staff, so old bills always point at a real record.
- A bill line keeps a **copy** of the service's description, so editing the service later never changes old bills.
- A bill belongs to the **financial year** of its date (1 April to 31 March): 31 Mar 2026 is in FY 2025-26, 1 Apr 2026 is in FY 2026-27.
- Each bill records who created it and who last edited it, and when.
- Deactivated staff lose access immediately, not when their login token expires.

See docs/decisions.md for the reasoning behind these choices.

Look and feel

- **Light, dark and system themes.** A three-way switch in the top bar (and on the login page): *device setting*, *light*, *dark*. The choice is remembered in the browser and applied before the page paints, so there is no white flash. Bills stay a white sheet of paper in dark mode, exactly as they print, so any logo reads correctly.
- **Built on the principles in emilkowalski/skills**: restrained motion (this is a tool used all day, so only buttons, fades for messages and colour changes move, and nothing animates on a keyboard action), buttons that press down slightly, hover effects only on devices that can hover, semi-transparent edges instead of solid borders, tabular figures for amounts, tighter heading spacing, and a stress test with worst-case data.
- **Phone friendly.** The Bills list and the bill form become card layouts on narrow screens, inputs are 16px so iPhones do not zoom in, tap targets are at least 44px on touch screens, and safe-area insets are respected.
- **Plain-language errors and enforced limits.** Forms stop you typing past a field's limit, and messages read like "Phone is too long (at most 20 characters)".

Client folders

For every client the server keeps a normal folder, named after the client, as a human-readable second copy:

```
Client Files/
  Rajesh Patel/
    client-details.txt        details and a list of the client's bills
    client-data.json          the same data in full, for other programs
    bills/INV-000007_2026-11-05.html     each bill, printable in any browser
```

- Updated whenever a client or bill is saved, and checked again every time the server starts.
- Two clients with the same name: the second folder gets the client number added, e.g. `Rajesh Patel (2)`.
- Renaming a client renames the folder. Deleting a client keeps the folder and marks its details file as deleted. Folders are never deleted by the software.
- Location: `CLIENT_FILES_DIR` in `server/.env` (default `server/client-files` in development, the folder chosen in the installer wizard when installed). Admins can see the location, the last error, and press **Rebuild all client folders now** under Settings.
- If the disk cannot be written to, bills still save and Settings shows the error.

Known limitations (ideas for v0.2)

- Bill numbers come from a database sequence, so a bill that fails to save can leave a gap in the numbers (GST invoice numbers normally must be consecutive).
- Issued bills can be edited. A real accounting workflow would cancel them or issue credit notes instead.
- GST is one flat rate (no CGST/SGST/IGST split, HSN/SAC codes or GST reports).
- No PDF or email export (the browser's Print works).
- The login rate limit is in memory, so it resets when the server restarts.

User guide

How to use Billing Software day to day. Anything marked **(admin)** is only available to administrators.

Contents

1. Logging in
2. Clients
3. Services
4. Creating a bill
5. Finding, viewing, editing and printing bills
6. Staff accounts (admin)
7. Organisation settings (admin)
8. Client folders on the PC
9. Backups and restoring
10. Light and dark theme
11. Troubleshooting

1. Logging in

Open the address your administrator gave you, for example `http://OFFICE-PC:4000`, and log in with your username and password. There is no sign-up: an administrator creates every account (see Staff accounts).

- After 10 wrong attempts for the same username from the same computer you are asked to wait about 15 minutes.
- A deactivated account stops working immediately, even if the person is still logged in.
- **Log out** is at the top right.

2. Clients

Clients in the top menu.

- **Add client:** name is required. Phone, email, city, address, PAN, GSTIN and **Referred by** are optional. PAN and GSTIN are saved in capital letters.
- **Search** by name, phone or the "referred by" text.
- **Edit** changes any detail. The client's *Files folder* name is shown while editing (see section 8).
- **Delete** hides the client but keeps all their bills. Tick **Show deleted** to see deleted clients and press **Restore** to bring one back. You cannot create a *new* bill for a deleted client, but their old bills stay visible and printable.

Two clients may share a name (for example two "Rajesh Patel"). The bill search and the client picker show their phone and city so you can tell them apart.

3. Services

Services in the top menu. A service is something you charge for, with a default price and an optional description.

- **Add service:** name and default price (in rupees, for example `2500` or `2500.50`) are required.
- The **description** you enter here is copied onto each bill line that uses the service, in smaller text under the title.
- Untick **Active** to stop a service being offered on new bills. Existing bills are not affected.
- Editing a service never changes bills that were already made.

4. Creating a bill

Bills > Create bill.

1. **Client:** type a name or phone number and pick the client from the list.
2. **Bill date:** defaults to today. The **financial year** (April to March, for example 2026-27) is shown beside the label.
3. **Items:** for each item either choose a **Service** (the item name, description and price are filled in for you) or leave it as *Custom item* and type them yourself.
    - **Item name** is the bold title printed on the bill.
    - **Description** is the smaller text printed under it. It is filled in from the service and you can change it for this bill only.
    - **Qty** is a whole number. **Price** is in rupees.
    - **+ Add item** adds another line; **Remove** deletes one.
4. The **totals** update as you type. If GST is switched on in settings, the GST line appears automatically.
5. **Notes** are optional and print at the bottom of the bill.
6. Press **Create bill**.

Each bill gets the next bill number (`INV-000001`, `INV-000002`, ...).

GST is fixed when the bill is created. If an administrator later changes the GST setting, old bills do not change.

Limits: a bill can have up to 100 items, and the items may add up to at most ₹100 crore. Fields stop you typing past their limits.

5. Finding, viewing, editing and printing bills

Bills in the top menu lists bills, newest first, 20 per page.

- **Search** by client name or bill number (for example `INV-000012`).
- **Financial year** dropdown: choose *FY 2025-26* to see only bills dated 1 April 2025 to 31 March 2026, or *All financial years*. A bill dated 31 March 2026 belongs to FY 2025-26; one dated 1 April 2026 belongs to FY 2026-27.
- Click a row to open the bill. On a phone each bill is shown as a card.

On a bill:

- **Print** prints the bill without the browser's own date, title and address lines. It always prints on white, even in dark mode. If your browser still adds a header or footer, untick **Headers and footers** in its print window once.
- **Edit bill** lets you change the client, date, items and notes. The GST rate stays the one the bill was created with. The bill records who created it and who last edited it, and when.

6. Staff accounts (admin)

Staff in the top menu.

- **Create account:** full name, username (3 to 30 characters: lowercase letters, numbers, dot, dash or underscore), a password of at least 8 characters, and a role.
  - **Staff** can use clients, services and bills.
  - **Admin** can also manage staff and settings.
- **Reset password** sets a new password for someone. Share it with them privately.
- **Deactivate** stops an account from working (it is not deleted, so their old bills still show who made them). **Reactivate** turns it back on.
- You cannot deactivate or demote your own account, so you can never lock everyone out by accident.

7. Organisation settings (admin)

Settings in the top menu.

- **Logo:** upload a PNG, JPEG or WebP image (up to 1 MB; large images are shrunk automatically). It prints at the top of every bill. SVG files are not accepted. A logo on a transparent or white background looks best. **Remove logo** takes it off.
- **Firm details:** name, address and GSTIN, printed on every bill.
- **Payment details:** account holder, bank, account number, IFSC, branch, UPI ID and free-text instructions, printed in a "please pay to" box so clients know where to pay. Anything left empty is not printed. These are your *current* details on every bill, so when your bank account changes, unpaid bills show the new one.
- **GST:** switch **Charge GST on new bills** on or off and set the rate. It only affects bills created afterwards.
- **Client folders:** shows where the folders are kept and whether the last update worked (see below).

Press **Save settings** to save the firm, payment and GST sections. The logo saves the moment you choose it.

8. Client folders on the PC

For every client the software keeps an ordinary Windows folder named after them, as a second readable copy of their records:

```
Client Files/
  Rajesh Patel/
    client-details.txt        their details and a list of their bills
    client-data.json          the same information in full, for other programs
    bills/INV-000007_2026-11-05.html     each bill, openable and printable in any browser
```

- Updated automatically whenever a client or bill is saved, and checked again every time the software starts.
- Two clients with the same name get `Rajesh Patel` and `Rajesh Patel (2)`. Renaming a client renames their folder.
- Deleting a client keeps their folder; the details file then says DELETED. The software never deletes these folders.
- Do not edit the files: they are rewritten whenever something changes.
- If it ever shows an error in Settings, check that the folder exists, the disk is not full, and then press **Rebuild all client folders now** (admin).

The location was chosen when the software was installed. When you set it up from the source code, it is set in `CLIENT_FILES_DIR` (see the architecture notes).

9. Backups and restoring

The database is backed up automatically when the software starts and then every 24 hours, keeping the 14 most recent in `C:\ProgramData\BillingSoftware\backups` (installed version).

Please copy that folder, and the client folders, to a USB drive or cloud drive regularly. Backups that live only on the same PC do not help if that PC fails.

To restore (installed version): open Command Prompt **as administrator** and run

```
"C:\Program Files\Billing Software\tools\restore.cmd"
"C:\ProgramData\BillingSoftware\backups\billing-YYYYMMDD-HHMMSS.tar.gz"
```

using the real file name. The current database is set aside, not deleted. Anything entered after that backup was made will not be in the restored copy. If you use a full PostgreSQL server instead of the built-in database, back it up with `pg_dump`.

10. Light and dark theme

The three small buttons at the top right choose **device setting**, **light** or **dark**. The choice is remembered in that browser. Bills always stay white, because they print on white.

11. Troubleshooting

Problem	What to try
Another computer cannot open the address	Check both are on the same network; the host PC is on and not asleep; try its IP address (`ipconfig` on the host) instead of its name; set the host's network to **Private** in Windows settings.

Problem	What to try
"Too many failed attempts"	Wait about 15 minutes, or ask an administrator to reset your password.
"Phone is too long (at most 20 characters)" or similar	The message names the field and its limit; shorten it.
"The bill total is too large"	Check the quantities and prices; a bill's items may add up to at most ₹100 crore.
A client folder is missing or out of date	Admin: **Settings > Rebuild all client folders now**.
Settings shows a client folder error	Check the folder exists, is writable and the disk is not full, then rebuild.
Bill numbers skip (for example 8, 9, then 34)	Numbers are taken before a save finishes and are not given back if it fails. This is a known limit.
The software will not start (installed version)	Look in `C:\ProgramData\BillingSoftware\logs`, or start "Billing Software" in Windows **Services**.

API reference

The server exposes a JSON API under `/api`. The React app is its only intended client, but it is plain HTTP and can be used from scripts.

Conventions

- **Base URL:** `http://localhost:4000/api` in development.
- **Authentication:** every route except `GET /api/health` and `POST /api/auth/login` needs `Authorization: Bearer <token>`. Tokens come from login and last 8 hours. The user is looked up on every request, so a deactivated user is locked out at once.
- **Roles:** `admin` and `staff`. Routes marked *admin* return `403` to staff.
- **Money** is always **integer paise** (1 rupee = 100 paise), in fields ending `_paise`. Example: ₹2,500.50 is `250050`.
- **Dates** are `YYYY-MM-DD`. Timestamps are ISO 8601.
- **Lists** are paged: `?page=1&limit=20` (limit 1 to 100, default 20) and return `{ "items": [...], "total": 57, "page": 1, "limit": 20 }`.
- **Errors** are `{ "error": "A sentence a person can act on" }`. Status codes:

Status	Meaning
400	Validation failed (the message names the field and the rule)
401	Not logged in, token expired, or account deactivated
403	Admin access required
404	Not found
409	A unique value already exists (for example a username)
413	Upload too large (logo over 1 MB)
415	Wrong file type for an upload
429	Too many failed logins (10 per username and address per 15 minutes)
500	Unexpected server error (details are in the server log, never in the response)

Health

Method and path	Description
`GET /api/health`	`{ "ok": true }`. No login needed.

Authentication

Method and path	Body	Result
`POST /api/auth/login`	`{ "username", "password" }`	`{ "token", "user": { id, username, full_name, role } }`
`GET /api/auth/me`		`{ "user": { ... } }`

Users (admin)

Method and path	Body	Notes
`GET /api/users`		`{ "items": [users] }`
`POST /api/users`	`username`, `password`, `full_name`, `role` (default `staff`)	201. Username: 3-30 chars of `a-z 0-9 . _ -` (lower-cased). Password 8-100 chars.
`PATCH /api/users/:id`	any of `full_name`, `role`, `is_active`, `password`	You cannot deactivate or demote yourself.

Clients

Method and path	Description
`GET /api/clients?q=&include_deleted=&page=&limit=`	Search name, phone and referred-by. Deleted clients are hidden unless `include_deleted=true`.
`GET /api/clients/:id`	One client.
`POST /api/clients`	Create (201).
`PUT /api/clients/:id`	Replace the editable fields.
`DELETE /api/clients/:id`	**Soft** delete (204). Bills are kept.
`POST /api/clients/:id/restore`	Undo a delete.

Client fields and limits: `name` (required, 150), `phone` (20), `email` (150, must look like an address), `address` (300), `city` (100), `pan` (10, upper-cased), `gstin` (15, upper-cased), `referred_by` (150). Empty text is stored as `null`. Responses also include `folder_name` (see client folders), `is_deleted`, `created_at`, `updated_at`.

Services

Method and path	Description
`GET /api/services?include_inactive=`	Active services unless `include_inactive=true`.
`POST /api/services`	Create (201).

Method and path	Description
`PUT /api/services/:id`	Replace. Existing bills are not changed.

Fields: `name` (required, 150), `description` (500), `price_paise` (integer, 0 to 100,000,000,000), `is_active` (default `true`).

Bills

Method and path	Description
`GET /api/bills/financial-years`	`{ "items": [{ "fy_start": 2026, "label": "2026-27" }] }`: the years that have bills (newest first), always including the current one.
`GET /api/bills?q=&client_id=&fy=&page=&limit=`	Newest first. `q` matches the client name or bill number. `fy` is the **start year** of an Indian financial year (April to March): `fy=2025` means 1 Apr 2025 to 31 Mar 2026.
`GET /api/bills/:id`	The bill with `items`, client details, `fy_start`, and who created and last edited it.
`POST /api/bills`	Create (201).
`PUT /api/bills/:id`	Replace the client, date, notes and items.

Body for create and replace:

```
{
  "client_id": 12,
  "bill_date": "2026-11-05",
  "notes": "Optional, up to 1000 characters",
  "items": [
    {
      "service_id": 3,
      "description": "Statutory Audit",
      "details": "Optional text printed under the title, up to 500 characters",
      "quantity": 1,
      "unit_price_paise": 2500000
    }
  ]
}
```

- `bill_date` is optional (today when omitted). `service_id` and `details` are optional. `description` is 1 to 300 characters; `quantity` is a whole number 1 to 100,000; `unit_price_paise` is 0 to 100,000,000,000.
- 1 to 100 items. The items may add up to at most 1,000,000,000,000 paise (₹100 crore), otherwise `400`.
- The client must exist and not be deleted (an existing bill may keep a client deleted since, but cannot be moved to one).
- **GST is a snapshot.** On create the current setting is read once and stored as `gst_rate_percent` and `gst_paise` on the bill. On replace the bill's own stored rate is used, never the current setting.
- `bill_no` is assigned on create (`INV-000001`, ...) and never changes.
- Totals are returned as `subtotal_paise`, `gst_paise`, `total_paise`, and each line has `line_total_paise`.

Settings

Method and path	Who	Description
`GET /api/settings`	any user	Firm, payment and GST settings, plus `has_logo` and `logo_version`.
`PUT /api/settings`	admin	Replaces **all** of the fields below (send the whole form).
`GET /api/settings/logo`	any user	The logo image. `404` if none. Needs the token, so fetch it with the header rather than an `<img src>`.
`PUT /api/settings/logo`	admin	The image as the **raw request body** with `Content-Type: image/png`, `image/jpeg` or `image/webp`. Up to 1 MB.
`DELETE /api/settings/logo`	admin	Removes the logo.

Settings fields and limits: `org_name` (150), `org_address` (300), `org_gstin` (15, upper-cased), `bank_account_holder` (150), `bank_name` (150), `bank_account_number` (30), `bank_ifsc` (15, upper-cased), `bank_branch` (150), `upi_id` (100), `payment_instructions` (500), `gst_enabled` (boolean, required), `gst_rate_percent` (0 to 100, required).

The logo's real type is read from the file's first bytes, not the `Content-Type` header. SVG is refused because it can contain scripts.

Client folders (admin)

Method and path	Description
`GET /api/client-files/status`	`{ enabled, dir, lastError, lastSync }`
`POST /api/client-files/rebuild`	Rewrites every client's folder from the database. Returns the count and the status.

Example

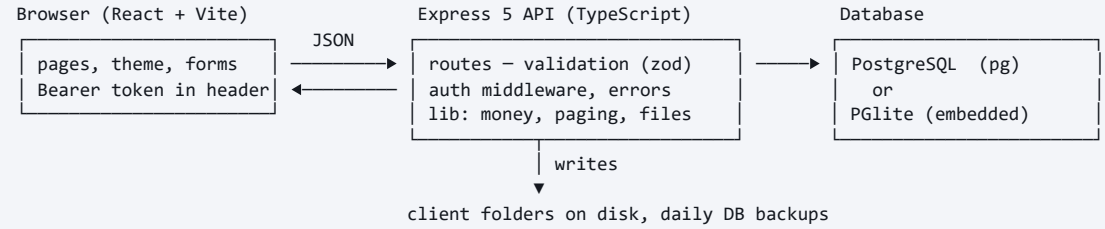
```
# log in
TOKEN=$(curl -s -X POST http://localhost:4000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"YOUR-PASSWORD"}' | node -pe
'JSON.parse(require("fs").readFileSync(0)).token')

# bills for financial year 2025-26 matching "patel"
curl -s "http://localhost:4000/api/bills?q=patel&fy=2025" -H "Authorization: Bearer $TOKEN"
```

Architecture and development notes

For the reasoning behind individual choices see decisions.md. For using the software see the user guide, and for the HTTP interface the API reference.

Overview



In production the Express server also serves the built React app (`client/dist`), so one process serves everything.

Repository layout

```
server/
  migrations/        numbered SQL files, applied in order at startup
  src/
    index.ts         startup: connect, migrate, backups, client folders, listen
    app.ts           builds the Express app and mounts routes
    config.ts        environment settings (loads .env, creates the login secret)
    db/index.ts      database adapter: PostgreSQL (pg) or PGlite, same interface
    backup.ts        daily database backup (embedded database only)
    lib/             money maths, paging helpers, error messages, client folders
    middleware/      authentication and admin check
    routes/          auth, users, clients, services, bills, settings, clientFiles
    scripts/         seed.ts (first admin), restore.ts (restore a backup)
  test/              Vitest + Supertest (PGlite in memory)
client/
  src/
    pages/           one file per screen
    components/      layout, theme switch, pager, payment box, logo mark
    api.ts           fetch wrapper with the token and uploads
    money.ts         rupee formatting, financial-year helpers, totals preview
    theme.ts         light / dark / system theme
    styles.css       design tokens and all styles
installer/           Inno Setup script, build script, wizard text, restore.cmd
.github/workflows/   ci.yml (tests) and release.yml (builds the installer)
docs/                these documents
  pdf/               PDF copies of every document (and one combined PDF)
  pdf-build/         the script that makes them: Markdown to HTML to PDF using Edge or Chrome
```

Data model

Table	Purpose
`users`	Accounts. `role` is `admin` or `staff`; `is_active` is the soft delete.
`clients`	Clients. `is_deleted` is the soft delete; `folder_name` is their folder on disk.
`services`	What the firm charges for. `is_active` hides one from new bills.
`bills`	One row per bill: number, client, date, notes, subtotal, `gst_rate_percent`, `gst_paise`, total, and who created and last edited it.
`bill_items`	The lines of a bill: `description` (title), `details` (description), quantity, unit price and line total.
`settings`	Key/value: firm details, payment details, GST on/off and rate.
`org_logo`	At most one row: the logo bytes and type.
`schema_migrations`	Which migration files have run.

`bill_no_seq` is the sequence that numbers bills.

Rules worth knowing:

- **Money is integer paise** everywhere. GST is computed with integer maths on basis points and rounds half up (`server/src/lib/money.ts`). The same function exists in `client/src/money.ts` for the live preview.
- **GST and service descriptions are snapshots** stored on the bill, so changing settings or a service never rewrites old bills.
- **The financial year is calculated, not stored**, from the bill date (April to March).
- **Soft delete** for clients, staff and services, so bills always point at real rows.

Migrations

`server/migrations/NNN_name.sql` files are applied in name order at startup inside a transaction each, and recorded in `schema_migrations`. To change the schema add a new numbered file; never edit one that has shipped.

Database adapter

`server/src/db/index.ts` exposes one small `Db` interface (`query`, `exec`, `tx`, `close`, optional `backup`) with two implementations chosen by `DATABASE_URL`:

- `postgres://...` uses the `pg` driver against a real PostgreSQL server.
- `pglite:<folder>` (or `pglite:memory`) uses PGlite, PostgreSQL compiled to WebAssembly. The installer uses it so nothing else has to be installed, and the tests use the in-memory form.

The driver is told to return `BIGINT`, `NUMERIC` and `DATE` as JavaScript numbers and plain `YYYY-MM-DD` strings.

Client folders

`server/src/lib/clientFiles.ts` writes a folder per client containing `client-details.txt`, `client-data.json` and an HTML copy of every bill (with the logo embedded and the payment details). It is called after each client or bill change, after settings or the logo change, and for every client at startup.

- One operation at a time (a queue), so renames and writes never race.
- Folder names are made safe for Windows; same-named clients get `(id)` added; renaming a client renames the folder.
- Folders are only created, renamed or refreshed, never deleted. Stale bill copies (date changed, bill moved to another client) are removed from the old place.
- A failure is logged and shown in Settings but never stops the request that triggered it.

Backups

For the embedded database `server/src/backup.ts` writes `billing-<timestamp>.tar.gz` into `<DATA_DIR>/backups` at startup (if the newest is over a day old) and every 24 hours, keeping 14. `server/src/scripts/restore.ts` loads one back (the service must be stopped; the old database folder is moved aside). A PostgreSQL server is backed up with `pg_dump` instead.

Security notes

- Passwords are hashed with bcrypt. Logins are rate limited per username and address (in memory, so it resets on restart).
- The user is loaded from the database on every request: deactivation is immediate.
- Search text is escaped so `%` and `_` are ordinary characters; all queries are parameterised.
- Text from users is escaped when written into the saved HTML bill copies.
- The logo accepts only PNG, JPEG and WebP, checked from the file's bytes, up to 1 MB; SVG is refused. It needs a login to fetch.
- The app is plain HTTP and meant for a private office network; do not expose it to the internet.

Configuration

Read from `server/.env` (copy `server/.env.example`) or the environment.

Variable	Meaning
`PORT`	Port to listen on (default 4000).
`DATABASE_URL`	`postgres://user:pass@host:5432/db` or `pglite:./.data` (default).
`JWT_SECRET`	Signs login tokens. Required in production unless `DATA_DIR` is set (then one is generated and kept).
`DATA_DIR`	Set by the installer: where the database, backups, logs and login secret live.

Variable	Meaning
`CLIENT_FILES_DIR`	Where client folders are written (default `server/client-files`, or `<DATA_DIR>/ClientFiles`).
`ADMIN_USERNAME`, `ADMIN_PASSWORD`, `ADMIN_FULL_NAME`	Used by `npm run seed` to create the first admin.
`NODE_ENV`	`production` for the installed service.

Development

```
cd server && npm install && cp .env.example .env && npm run seed && npm run dev    # API on :4000
cd client && npm install && npm run dev                                            # app on :5173
```

Command (in `server/`)	What it does
`npm run dev`	API with auto-restart.
`npm run typecheck`	TypeScript check.
`npm test`	All tests (in-memory database, no setup).
`npm run build`	Compile to `server/dist`.
`npm run seed`	Create the first admin from `.env`.

`npm run build` in `client/` type-checks and builds `client/dist`.

Tests

56 tests in `server/test/`: money maths, authentication and roles, staff management, clients, services, bills (GST snapshot, financial years, search and paging, totals ceiling), validation messages, the logo upload (including SVG and fake images), and the client folders (names, renames, moves, deletes, escaping, rebuild, disk failure). CI runs them on every pull request.

Front-end conventions

- All colours, shadows and radii are CSS variables in `client/src/styles.css`; light, dark and system themes redefine them.
- A bill is a "paper" (`.paper`): always white, even in dark mode.
- Hover styles are wrapped in `@media (hover: hover) and (pointer: fine)`; inputs are 16px on touch screens.
- Only `transform` and `opacity` animate, with one custom ease-out curve and no `transition: all`.
- Every form field enforces the same length limit as the server.

Installer and releases

`installer/billing.iss` (Inno Setup) builds `BillingSoftware-Setup-<version>.exe`. It bundles a private Node.js, the compiled server with production dependencies, the built React app and WinSW (which runs the server as a Windows service).

The wizard shows an information page, asks for the port, the client-folder location and the first administrator, shows a full summary, then: writes `server\.env`, creates the administrator (the password goes through a temporary file, never a command line), adds a firewall rule for private and domain networks, installs and starts the service, and creates shortcuts. Re-running it upgrades in place and keeps all data. The uninstaller asks before deleting data, and never deletes client folders.

`installer/build.ps1` assembles the pieces into `installer/staging` (options: `-NodeExe` to use an existing `node.exe`, `-SkipService` to skip the WinSW download). `.github/workflows/release.yml` runs the tests, builds, compiles the installer with Inno Setup and attaches it to a GitHub Release when a tag such as `v0.1.0` is pushed:

```
git tag v0.1.0 && git push origin v0.1.0
```

The installer is not code-signed, so Windows shows an "unknown publisher" warning.

Known limitations

- Issued bills can be edited (no cancel or credit-note workflow yet).
- Bill numbers come from a database sequence, so a failed save can leave a gap; GST invoices normally need consecutive numbers.
- GST is a single flat rate: no CGST/SGST/IGST split, HSN/SAC codes or GST reports.
- No PDF or email export (the browser's Print works).
- The login rate limit is in memory.
- The installer wizard and service have not yet been compiled and run end to end (the first release build will do so).
- The app has not yet been tried on a real phone.

Decision records

Short notes on decisions that shape the project: what was decided, why, and what follows from it.

1. Bill search

Decision. One search page. The user types a client name or a bill number and gets that client's bills as a list, newest first, with the bill date and bill number shown prominently.

Why. An accountancy firm can have hundreds of bills per client. Finding one by name or number is the common task.

Consequences.

- Results are paginated (20 per page) instead of loading everything.
- `bills` has indexes on `(bill_date DESC, id DESC)` and `client_id` so search and sorting stay fast.
- Two clients can share a name, so each result also shows the phone and city.
- Bill number is unique (`INV-000001` , from a database sequence); client name is only a search term.

2. Bills can be edited in v0.1

Decision. Any bill can be edited for now. This may change in a later version.

Why. It keeps v0.1 simple, and the firm can fix typos.

Consequences.

- Known risk: in real accounting, an issued bill is normally cancelled or corrected with a credit note.
- Safeguard: every bill stores `created_by` , `created_at` , `updated_by` and `updated_at` , so we can always see who changed it and when. Credit notes can be added later without losing data.

3. GST is built, but off by default

Decision. GST support exists and an admin turns it on with a toggle in organisation settings (with a rate).

Why. The firm does not charge GST yet, but will.

Consequences.

- Tax is **snapshotted on each bill** (`gst_rate_percent` , `gst_paise`). Changing the setting never alters old bills.
- Editing a bill recalculates using *that bill's* stored rate, not the current setting.
- GST uses integer maths on basis points and rounds half up.
- Settings are admin-only to change; every user can read them (the bill form needs the rate).
- Not built yet: CGST/SGST/IGST split, HSN/SAC codes, GSTR reports.

4. Admin creates staff accounts

Decision. No public sign-up. The admin creates accounts. Roles are `admin` and `staff`.

Why. This is an internal tool for one firm. Anyone on the internet must not be able to create an account.

Consequences.

- The first admin is created by a seed script (`npm run seed`) using values from `server/.env`.
- Staff are deactivated, not deleted, so old bills still show who created them.
- The user is loaded from the database on every request, so deactivation takes effect immediately.
- An admin cannot deactivate or demote their own account (prevents locking everyone out).
- Repeated failed logins are rate limited per username and IP address.

5. Money is stored as integer paise

Decision. Every amount is a whole number of paise in the database, API and server code.

Why. Floating point cannot represent values like 0.1 exactly (`0.1 + 0.2` is `0.30000000000000004`), which is unacceptable in billing.

Consequences.

- The API takes and returns `*_paise` fields; only the UI converts to rupees.
- The UI converts typed amounts by splitting the text, never by multiplying a float.

6. Soft delete for clients, services and staff

Decision. Records are flagged (`is_deleted` / `is_active`) instead of removed.

Why. Bills refer to clients, services and users. Removing them would break old bills or lose history.

Consequences.

- Deleted clients are hidden by default but can be shown and restored.
- A bill may keep belonging to a deleted client, but new bills cannot be created for one.

7. Windows installer: one office PC as the server, embedded database

Decision. A Setup.exe wizard (Inno Setup) installs the app on one PC as a Windows service. Other staff use it from a browser over the office network. The installer bundles Node.js and uses PGlite (Postgres compiled to WebAssembly) as the database, so nothing else has to be installed. It is built and published by GitHub Actions when a version tag is pushed. Windows only.

Why. The firm is small and non-technical. Installing PostgreSQL separately is the step most likely to go wrong.

Consequences.

- All data lives in `C:\ProgramData\BillingSoftware`, separate from the program, so upgrades never touch it.

- The embedded database allows only one process to open it, so only the service uses it. Backups are taken by the server itself (at startup and every 24 hours, last 14 kept), and `restore.cmd` is used to restore one.
- Backups on the same PC do not protect against that PC failing; the wizard tells the user to copy them elsewhere.
- The login secret is generated once on first start and stored in the data folder.
- The installer is not code-signed, so Windows shows an "unknown publisher" warning.
- The firewall rule covers private and domain networks only. The app uses plain HTTP and is meant for the office LAN, not the internet.
- Larger firms can still point the same server at a real PostgreSQL server with `DATABASE_URL`.

9. A readable folder per client, kept on the PC

Decision. The server writes a folder for every client (named after them) containing a details file, a JSON copy and an HTML copy of every bill. It is kept up to date automatically and rebuilt at startup.

Why. The firm wants its client records as ordinary files it can open, copy or send, independent of the software.

Consequences.

- It is a second copy, not the source of truth: the database is. The files are rewritten, so editing them has no effect.
- Folder names are made safe for Windows (illegal characters become `-`, reserved names and trailing dots are handled). Duplicate names get the client number added. The chosen folder is stored in `clients.folder_name`.
- Folders are only ever created, renamed or refreshed, never deleted. A bill that moves to another client, or whose date changes, has its old copy removed from the old place so the folder never shows stale data.
- File problems never block saving a bill; they are logged and shown to admins in Settings.
- Like the database backups, these folders live on the same PC, so the firm should copy them elsewhere regularly.

10. Financial years and service descriptions on bills

Decision. Bills are grouped by Indian financial year (1 April to 31 March), with a dropdown on the Bills page. A bill line carries a copy of the service's description.

Why. Accountants work and file by financial year. The description explains what the client is paying for.

Consequences.

- The FY is calculated from the bill date, not stored, so it can never disagree with the date.
- The description is copied onto the line when the bill is made (editable per bill), so changing the service later does not rewrite bills already issued, the same rule as the GST snapshot.

11. Logo and payment details on every bill

Decision. An admin sets the firm's logo and payment details (account holder, bank, account number, IFSC, branch, UPI ID, free-text instructions) in Settings. They print on every bill and in the client-folder copies. Anything left empty is not printed.

Why. Clients need to know where to pay.

Consequences.

- They are the firm's *current* details, shown on every bill (not frozen per bill like GST), because when the firm changes its bank account, bills that are still unpaid should show the new one. Saving them refreshes the client-folder copies.
- The logo is stored in the database (table `org_logo`), so it is included in the normal backups.
- Only PNG, JPEG and WebP are accepted, up to 1 MB, and the real file type is checked from the file's contents. SVG is refused because it can contain scripts. The browser shrinks large images before uploading.
- Only admins can change these; every logged-in user can view them. The logo is not public: it needs a login to fetch.
- The payment details are plain text typed by the admin. The software does not take payments or check the account.

12. Themes and design principles

Decision. The app has light, dark and "follow the device" themes, a token-based design system (all colours, shadows and radii are CSS variables) and follows the guidance in github.com/emilkowalski/skills.

Why. Staff use the software for hours a day, so comfort, clarity and not being slowed down matter more than flourish.

Consequences.

- **Motion is deliberately minimal.** Used-all-day actions (opening a list, typing, keyboard shortcuts) never animate. Only press feedback (160 ms, scale 0.97), colour changes (150 ms) and message fades (200 ms) move, with one strong ease-out curve and never `ease-in` or `transition: all`. Reduced-motion users get the fades without the movement.
- **A bill is a document.** It is always a white sheet (`.paper`), in dark mode too, because it prints on white and logos with dark text on a transparent background would vanish on a dark card.
- **Worst-case data is a test.** Long names (up to the real 150-character limit), emoji and non-Latin names, one-letter names, maximum-length descriptions and 12-item bills were loaded into a throwaway database and checked at 320 px wide, in both themes. Amounts and dates are never truncated; text wraps; badges never wrap.
- **The frontend and backend agree on limits.** Every field enforces its server limit as you type.
- **Bill totals have a ceiling.** A subtotal above ₹100 crore is refused, because quantity x price can otherwise exceed what can be added exactly.
- **Not verified:** the guidance says emulation cannot reproduce sticky hover, safe areas or the on-screen keyboard. Please also try the app on a real phone.

8. Plain SQL and a small database adapter

Decision. SQL migrations and hand-written queries (no ORM yet), behind a tiny `Db` interface.

Why. Learning goal: see exactly what the database is asked to do. The interface also lets the tests run on PGlite (Postgres in WebAssembly) with no database to install, while production uses the real `pg` driver.

Consequences.

- Migrations live in `server/migrations/` and run in order at startup.
- An ORM can be introduced later without changing how the app is structured.